

Testarea Sistemelor **Software**

Sistem Gestione Cinema

Echipa: Lungu Bogdan Cosmin & Lascu Daniel

Configurație Sistem & Obiective



Stack Tehnologic

Python 3.14.2
pytest 9.0.3
mutmut 2.5.1



Obiectiv SUT

Metoda **calculate_price**: logica de preț bazată pe vârstă, student, zi, rând și loialitate.



Etape Testare

Testare Funcțională (Black-Box),
Testare Structurală (White-Box) și
Mutation Testing.

Testare Funcțională (Black-Box): Inputs

Equivalence Classes

-  **Day:** Weekdays (1-5), Weekend (6-7)
-  **Row:** Standard (0-3, 7-9), VIP (4-6)
-  **Age:** Standard (0-64), Senior (65+)
-  **Loyalty Points:** No Disc (0-50), Disc (51+)

BVA (Boundary Value Analysis)

-  **Day:** {0, 1, 5, 6, 7, 8}
-  **Row:** {-1, 0, 3, 4, 6, 7, 9, 10}
-  **Age:** {-1, 0, 64, 65}
-  **Loyalty:** {-1, 0, 50, 51}

Testare Structurală

White-Box Strategies

🔑 Statement Coverage:

Instrucțiuni executate.

🔑 Decision Coverage:

Ramuri decizionale.

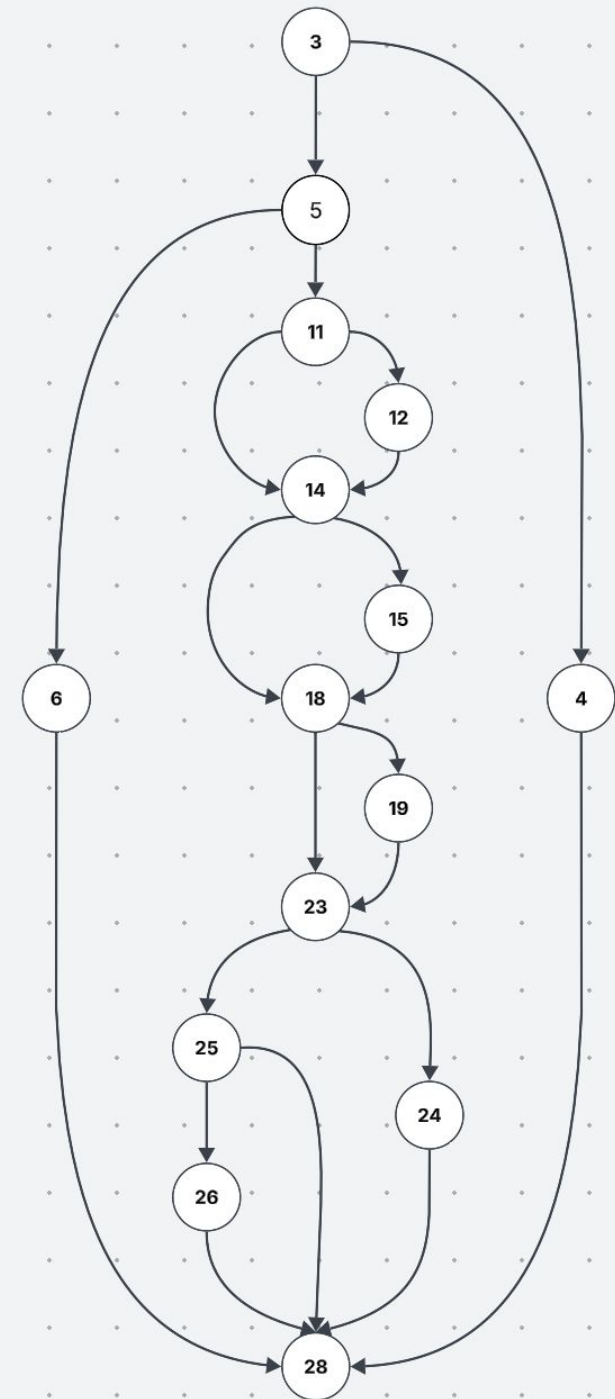
🔑 Condition Coverage:

Condițiile dintr-o decizie.

🔑 Independent Paths:

Căile McCabe.

```
1 def calculate_price(self, age, is_student, day, row, loyalty_points):
2     """calculeaza pretul biletului bazat pe zi, varsta, statut de student si rand"""
3     if day not in DAYS:
4         raise ValueError("Zi invalida (1-7)")
5     if not (0 <= row <= 9):
6         raise ValueError("Rand invalid (0-9)")
7
8     current_price = BASE_PRICE
9
10    # locuri VIP (4, 5, 6)
11    if 4 <= row <= 6:
12        current_price += 15.0
13
14    if loyalty_points > 50:
15        current_price *= 0.9
16
17    # scumpire weekend
18    if day >= 6:
19        current_price *= 1.1
20
21    # reducere (prioritate pensionar >= 65 ani)
22    # 2 tipuri de reduceri: pensionar/student
23    if age >= 65:
24        current_price *= 0.9
25    elif is_student and day <= 5:
26        current_price *= 0.85
27
28    return round(current_price, 2)
```



Mutation Testing

Pentru generarea de mutanti am folosit mutmut. Am generat mutanti pentru tot main.py.

Procesul de mutation testing a presupus 2 treceri:

Trecerea 1:

1. Generam mutanti cu mutmut si incercam sa ii omoram cu suita de teste functionale si structurale.
2. Identificam mutantii care supravietuiesc si scriem teste tintite in mutation_testing.py.

Trecerea 2:

1. Generam mutanti si rulam suita de teste care include si mutation_testing.py. Se poate observa ca numarul de mutanti care supravietuiesc scade.

Motivul pentru care supravietuiesc asa de multi mutanti este faptul ca acestia au fost generati pe tot fisierul main.py, nu doar pe functia noastra in scop (aflata in scopul proiectului).

Mutation testing report

Killed 45 out of 147 mutants

File	Total	Skipped	Killed	% killed	Survived
main.py	147	0	45	30.61	102

Mutation testing report

Killed 56 out of 147 mutants

File	Total	Skipped	Killed	% killed	Survived
main.py	147	0	56	38.10	91

Rezultate Testare

```
PS D:\Facultate\An4_sem2\TSS\ProjectFinal> python -m coverage run --branch -m pytest functional_testing.py structural_tests.py mutation_tests.py
===== test session starts =====
platform win32 -- Python 3.12.7, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\Facultate\An4_sem2\TSS\ProjectFinal
plugins: anyio-4.9.0
collected 23 items

functional_testing.py ... [ 13%]
structural_tests.py ..... [ 34%]
mutation_tests.py ..... [100%]

===== 23 passed in 0.10s =====
```

```
PS D:\Facultate\An4_sem2\TSS\ProjectFinal> python -m coverage report -m
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
functional_testing.py	37	1	10	1	96%	85
main.py	89	61	40	1	33%	45-51, 54-59, 62-123, 126
mutation_tests.py	55	0	0	0	100%	
structural_tests.py	33	0	0	0	100%	
TOTAL	214	62	50	2	67%	

AI Raport

Pentru testele generate de AI, am folosit Gemini.

Acesta a fost prompt-ul:

main

<> PY

Salut, am de facut Unit Testing pentru metoda "calculate_price" din fisierul main.py. Am de facut testare functionala, structurala si mutation testing. Te rog sa respecti urmatoarele cerinte:

Ilustrați strategiile de generare de teste prezentate la curs (partiționare în clase de echivalență, analiza valorilor de frontieră, acoperire la nivel de instrucțiune, decizie, condiție, circuite independente, analiză raport creat de generatorul de mutanți, teste suplimentare pentru a omorî 2 dintre mutanții neechivalenți rămași în viață) pe exemple proprii (create de echipă).

AI Raport

Testarea structurala si functionala acopera tot modului, lucru relevat de coverage analisys:

Pe parte de testare functionala, AI-ul a facut BVA destul de bine, in schimb nu a testat toate clasele de echivalenta.

In cazul mutation testing, nu a generat mutanti cu mutmut, ci doar a imaginat 3 muntanti posibili pentru care a generat 3 teste. Mutantii erau plauzibili, unul dintre ei chiar supravietuia in analiza de mutanti facuta de noi.

```
PS D:\Facultate\An4_sem2\TSS\ProjectFinal> coverage run -m unittest discover -s .\AI_testing
.....
-----
Ran 23 tests in 0.014s
```

```
OK
PS D:\Facultate\An4_sem2\TSS\ProjectFinal> python -m coverage report -m
Name                               Stmts  Miss  Cover   Missing
-----
AI_testing\test_functional.py       35      1    97%
AI_testing\test_structural.py       35      1    97%
main.py                             89     61    31%  45-51, 54-59, 62-123, 126
-----
TOTAL                               159     63    60%
```

```
PS D:\Facultate\An4_sem2\TSS\ProjectFinal> python -m unittest .\AI_testing\mutation_ai_testing.
py .\AI_testing\test_functional.py .\AI_testing\test_structural.py .\AI_testing\mutation_ai_testing.py
.....
-----
Ran 23 tests in 0.005s
```


Întrebări?

Vă mulțumim pentru atenție!

Lungu Bogdan Cosmin | Lascu Daniel